

AsyRand: Asynchronous Distributed Randomness Beacon with Reconfiguration

Liang Zhang, *Member, IEEE*, Tao Liu, Haibin Kan*, *Member, IEEE*, Jiheng Zhang*

Abstract—Distributed randomness beacon protocols, which continuously generate publicly verifiable randomness values, are crucial for many applications. Recently, there have been many approaches, such as Hydrand (S&P’20), SPURT (S&P’22), OptRand (NDSS’23) and GRandLine (CCS’24), based on publicly verifiable secret sharing (PVSS) to implement beacon protocols. However, two key challenges remain unresolved: asynchrony and reconfiguration. In this paper, we propose the AsyRand beacon protocol to address these challenges. We incorporate a producer-consumer model to decouple the distribution and reconstruction of PVSS secrets. Parties continuously produce and distribute new PVSS commitments, which are the encrypted shares and the proofs. Meanwhile, all parties store received commitments using first-in-first-out queues and collectively consume each commitment to recover the corresponding secret for beacon generation. To achieve asynchronous consensus, we employ reliable broadcast for distribution and apply t -validated asynchronous Byzantine agreement for reconstruction. To achieve reconfiguration, honest parties can collectively remove a faulty party if his queue remains empty for an extended duration, and a new party can join the system using reliable broadcast. We also introduce a novel PVSS scheme based on Sigma protocol and Fiat-Shamir heuristic, which is of independent interest. Consequently, AsyRand maintains state-of-the-art complexity with $O(n^2)$ communication complexity, $O(n)$ computation complexity, and $O(n)$ verification complexity while achieving asynchrony and reconfiguration. Experimental results highlight the performance of AsyRand compared to existing works.

Index Terms—Distributed Randomness Beacon, PVSS, Reconfiguration, Asynchrony

I. INTRODUCTION

Distributed randomness beacon protocols are designed to generate trustworthy and verifiable random values at intervals [1], [13], [6], [12], [34], [37], [35]. Beacons have a broad spectrum of applications, including secure multiparty computation [2], consensus protocols [6], [18], anonymous communication [8], [7], blockchain sharding [3], [4] and Byzantine agreement protocol [5] in distributed systems. The key properties of a beacon [13], [12], [34] are as follows:

- **Liveness/Availability:** All honest parties continuously proceed to generate new random output for every epoch.

Liang Zhang and Jiheng Zhang are with Department of Industrial Engineering and Decision Analytics, Hong Kong University of Science and Technology, Clear Water Bay, Kowloon, Hong Kong. (e-mail: scottzhang@ust.hk, jiheng@ust.hk)

Tao Liu is with School of Cyberspace Security (School of Cryptology), Hainan University, Haikou, 570228, China. (e-mail: jb8181632@gmail.com)

Haibin Kan is with School of Computer Science, Fudan University, Shanghai 200433, China, Shanghai Engineering Research Center of Blockchain, Shanghai 200433, China and Yiwu Research Institute of Fudan University, Yiwu City 322000, China. (e-mail: hbkan@fudan.edu.cn)

*Corresponding author

- **Bias resistance:** Each randomness must be unbiased and uniformly distributed.

- **Unpredictability:** No party should be able to predict a beacon’s value before it is generated.

- **Public verifiability:** Any party can verify that the beacon values are correctly generated following the protocol.

- **Responsiveness:** Beacon values are delivered at the speed of the actual network.

However, ensuring these properties in an asynchronous network while supporting reconfiguration remains an open challenge [10]. An asynchronous network allows messages to experience arbitrary delays, mirroring real-world network conditions [28], [48], [52]. Reconfiguration [37], [33], [45], the process of updating parties in distributed systems, is essential for removing faulty parties and adding external parties.

A. Our Approach in a Nutshell

We adopt publicly verifiable secret sharing (PVSS) (cf. Section II-A), the producer-consumer model (cf. Section II-B), reliable broadcast (RBC, cf. Section II-C) and validated asynchronous Byzantine agreement (t -VABA, cf. Section II-D) to enable asynchrony and reconfiguration in implementing the proposed AsyRand beacon.

PVSS We use PVSS as the underlying cryptographic primitive in secrets hiding and revealing, where the secrets serve as the seeds for beacon values generation. In PVSS, a dealer divides a secret into encrypted shares, each paired with a validity proof, forming a commitment that anyone can publicly verify. The dealer distributes the commitment to shareholders, who decrypt and validate their shares using the proofs. A subset of these decrypted shares, which are also publicly verifiable, can reconstruct the dealer’s secret.

Producer-Consumer Model We leverage the producer-consumer model to manage PVSS commitments asynchronously. Each party, playing the role of PVSS dealer, continuously and independently produces PVSS commitments and distributes to all parties. All parties, as shareholders, receive and collectively process these commitments to generate beacon values as follows. Each party locally maintains a collection of first-in-first-out (FIFO) queues $ComQ$, where $ComQ[i]$ stores party i ’s PVSS commitments. PVSS commitments are publicly verifiable, ensuring that no invalid commitments are added to these queues. Define an epoch as the period in which all parties consume a PVSS commitment. In each epoch e , the objective is to generate a new beacon value R_e . In epoch e , a leader, say party L , is randomly elected using the beacon value R_{e-1} from the previous epoch $e - 1$. All parties reconstruct leader

TABLE I: Comparison of PVSS-based beacon protocols.

Protocol	Network	Liveness	Comm.	Unpred.	Bias-ris.	Comp.	Verif.	Reconfig.	Resp.	Adversary
RandHerd [13]	syn.	✗	$O(c^2 \log n)$	1	✗	$O(c^2 \log n)$	$O(1)$	✗	✗	static
SCRAPE [15]	syn.	✓	$O(n^3)$	1	✓	$O(n^2)$	$O(n^2)$	✗	✗	static
HydRand [12]	syn.	✓	$O(n^2)$	$f + 1$	✓	$O(n)$	$O(n)$	✗	✗	static
GRandomPiper [33]	syn.	✓	$O(n^2)$	$f + 1$	✓	$O(n^2)$	$O(n^2)$	✓	✗	adaptive
SPURT [34]	semi-syn.	✓	$O(n^2)$	1	✓	$O(n)$	$O(n)$	✗	✓	adaptive
OptRand [37]	syn.	✓	$O(n^2)$	1	✓	$O(n)$	$O(n)$	✓	✓	static
GRandomLine [35]	syn.	✓	$O(n^2)$	1	✓	$O(n)$	$O(n)$	✗	✓	adaptive
AsyRand	asyn.	✓	$O(n^2)$	$f + 1$ †	✓	$O(n)$	$O(n)$	✓	✓	mobile‡

Unpred. depicts how many future epochs an adaptive rushing adversary can predict. In HydRand [12], GRandomPiper [33] and ours, a worst case exists with little probability where $l(\leq f)$ colluding malicious parties become leaders consecutively. In this occasion, the colluding parties can predict l future beacon values. Randomness is fully unpredictable beyond epoch $e + f$, due to the presence of at least one honest leader in any consecutive $f + 1$ epochs.

†: The unpredictability might be broken in AsyRand in the removal or joining process, as analyzed in Section V-A and Section V-B.

‡: We enable reconfiguration, meaning parties can join or leave. Thus, we assume a mobile adversary model and $f < n/3$ holds whenever n changes [50].

L 's secret gs by consuming the leader's first commitment in $ComQ[L]$. Then, the beacon randomness value R_e of epoch e is calculated as $R_e = Hash(R_{e-1}, gs)$, where $Hash$ is a hash function.

Asynchronous Consensus To guarantee consistency of PVSS commitments in asynchronous network, we use RBC for producing PVSS commitments and t -VABA for consuming them. Each party, as PVSS dealer, continuously leverages the RBC protocol to achieve consensus on new PVSS commitments among all parties. All parties, as shareholders, only add PVSS commitments that have reached agreement to their local queues. We design a new protocol t -VABA to enable parties, each holding a decrypted share, to collaboratively recover the leader's secret gs in each epoch. Upon completion of t -VABA, the beacon value R_e is successfully delivered.

Achieving Reconfiguration AsyRand supports reconfiguration by allowing parties to add or remove parties dynamically. On the one hand, honest parties can monitor the status of the queues to judge whether a party is faulty or not. They can automatically trigger removal of party i if $ComQ[i]$ is empty for an extended duration, since party i may have become faulty quietly. On the other hand, a new party θ (outside the system) can be added via proposing his PVSS commitment via a RBC protocol. Once the RBC protocol reaches agreement, each party puts θ 's PVSS commitment into its local queue $ComQ[\theta]$.

B. Related Works

Many approaches have been explored to build distributed randomness beacon protocols. Heuristically, public random numbers can be obtained as a byproduct of Bitcoin's PoW consensus [11]. Other cryptographic primitives are also employed as the underlying tools of beacon protocols, such as verifiable randomness function (VRF) [18], [9], verifiable delay function (VDF) [22], [38], threshold signatures [19], verifiable secret sharing (VSS) [32], [33], [45] and publicly verifiable secret sharing (PVSS) [15], [12], [34], [35]. Some beacon protocols [19], [20], also necessitate distributed key generation (DKG) [20], [24], [25], [21], [35] during the initial setup. Summarily, these primitives are leveraged to achieve a commit-and-reveal paradigm. In the committing phase, distributed parties introduce private randomness or entropy and

broadcast the corresponding commitment to others; In the revealing phase, the random value is uncovered and the final beacon value is calculated.

In the context of constructing a commit-and-reveal paradigm, PVSS schemes [42], [23], [39], [15], [16] are widely adopted as they do not require a private communication channel and can identify faulty parties. RandHerd [13] divides parties into c -size subgroups to achieve scalability with communication and computation complexity $O(c^2 \log n)$ at the cost of higher liveness failure probability. The verification complexity is $O(1)$ due to the use of collective signing with cryptographic multisignatures. In Ouroboros [6] and SCRAPE [15], n PVSS commitments from all parties are published for calculating a beacon value, resulting in $O(n^3)$ communication complexity. Moreover, SCRAPE leverages an optimized PVSS scheme, which reduces its computation and verification complexities to $O(n^2)$. HydRand [12], GRandomPiper [33] and AsyRand lower the complexity by randomly choosing leaders in each epoch. In a synchronous network, HydRand and GRandomPiper tolerates $1/3$ and $1/2$ faulty parties, respectively. SPURT [34] improves communication efficiency by aggregating PVSS commitments based on SCRAPE PVSS within a semi-synchronous network. However, SPURT weakens the liveness of the beacon protocol. If the leader of an epoch is faulty, the scheme allows parties to output a bot symbol instead of a randomness value. OptRand [37] further aggregates PVSS commitments and can tolerate up to $1/2$ faulty parties, incorporating the advantages of SPURT and GRandomPiper. However, OptRand assumes a synchronous network model. GRandomLine [35] runs with an optimized DKG in the pre-processing phase, which is based on a recursive aggregable PVSS. Both GRandomPiper [33] and OptRand [37] facilitate reconfiguration in a synchronized network. Rondo [45] (based on batched VSS and assuming semi-synchronous network) enables reconfiguration by requiring joining or leaving parties to actively submit online proposals.

Byzantine agreement (BA), Byzantine fault tolerance (BFT) or state machine replication (SMR) [26], [27] are commonly employed to achieve continuous agreement among distributed parties. Thus, the combination of PVSS primitive with BA-, BFT- or SMR-based consensus is widely employed to implement beacon protocols [13], [12], [34], [33], [37], [35].

Table I compares the PVSS-based beacon protocols. VSS-based beacon protocols are not included, as they require private

communication channels.

C. Contributions

- We propose AsyRand, a new PVSS-based beacon protocol for asynchronous network, by leveraging a producer-consumer model, RBC and t -VABA protocols. Each party produces PVSS commitments continuously and independently. Meanwhile, all parties collectively consume each commitment to deliver beacon value in every epoch. The production and consumption of commitments are guaranteed to be consistent via RBC and t -VABA protocols, respectively.

- Our AsyRand protocol supports reconfiguration. An external party can request to join by producing a valid PVSS commitment and distributing it using RBC, and all honest parties will accept this party upon RBC agreement. Each party can detect faulty parties by locally monitoring the state of queues. If the queue corresponding to a party remains empty for an extended duration, all honest parties will trigger the removal of this party.

- We propose t -VABA, a novel Byzantine agreement protocol, where the agreement value may be none of parties' inputs. Instead, the agreement value is collectively determined by any t honest parties. In AsyRand, t -VABA is applied for honest parties to deliver beacon values and to remove faulty parties.

- As of independent interests, we design a new PVSS scheme maintaining state-of-the-art complexity by leveraging the Sigma protocol [29] and Fiat-Shamir heuristic [30] to achieve non-interactive zero knowledge (NIZK) proofs. Numerical results show that our PVSS scheme performs well.

D. Limitations

The producer-consumer model enables distributed parties to handle PVSS commitments asynchronously. However, liveness is compromised if a party ceases to produce, resulting in no further PVSS commitments being available for consumption. To address this, we design a removal process that allows the system to adapt to failures. Given the FLP impossibility problem [36], AsyRand incorporates a rollback mechanism to coordinate the consumer and removal processes. Similarly, the joining process may also require rollback under adverse network conditions. A complete and robust solution to this challenge is left as future work. Promising directions include assuming an asynchronous binary agreement protocol [51] to prevent forks, or operating under a semi-synchronous network model to ensure termination and avoid rollback.

II. PRELIMINARIES

A. Publicly Verifiable Secret Sharing (PVSS)

A publicly verifiable secret sharing (PVSS) [15], [17], [43] enables a dealer to share secrets among distributed shareholders in a publicly verifiable manner. Particularly, the dealer shares a secret $gs = g^s$ among n shareholders $\mathcal{P} = \{P_1, \dots, P_n\}$, where $s \in \mathbb{Z}_p$. A PVSS scheme consists of the following five phases:

- 1) $(\{sk_i, pk_i\}) \leftarrow \text{PVSS.Setup}(\lambda, t, n)$ Each shareholder P_i generates a key pair (pk_i, sk_i) .

- 2) $(C, \pi) \leftarrow \text{PVSS.Share}(s, \{pk_i\})$ The dealer divides the secret into n shares. Each share is encrypted into C_i and all encrypted shares are accompanied by NIZK proofs. Then the dealer publishes the PVSS commitment, i.e., $(C = \{C_i\}, \pi)$.
- 3) $bool \leftarrow \text{PVSS.Verify}(C, \pi)$ Any external verifier can check whether the dealer has honestly shared a secret given (C, π) .
- 4) $D_i \leftarrow \text{PVSS.PreRecon}(C_i, sk_i)$ Each shareholder decrypts his encrypted share to obtain decrypted share D_i .
- 5) $gs \leftarrow \text{PVSS.Recon}(C, \{D_i\}_{i \in T})$ With a set of correct decrypted shares T ($|T| \geq t$), the secret value $gs = g^s$ can be reconstructed. Note that incorrect decrypted shares $\{D_i\}_{i \in T}$ can be detected and be abandoned.

A PVSS scheme satisfies the properties of **correctness**, **public verifiability** and **IND1-Secrecy**.

- **Correctness** Correctness ensures that at least t shareholders follow the protocol, the reconstructed secret will be identical to the original secret g^s .

- **Public Verifiability** Public verifiability allows anyone to verify the correctness of the encrypted shares from the dealer. Besides, the decrypted shares should also be publicly verifiable in the reconstruction phase.

- **IND1-Secrecy** IND1-Secrecy guarantees that an adversary, given any $t - 1$ secret keys $\{sk_i\}$ and the public information, learns no information about the secret gs .

B. Producer-Consumer Model

Producer-consumer model is a coordination pattern where a producer generates data items into a shared buffer while a consumer processes them, enabling asynchronous workflows. The buffer can be implemented using a FIFO queue, which ensures that data is consumed in the same order it is produced. A queue usually has a fixed length. Let $put(\cdot)$ and $get()$ represent the operations for adding data to the queue and retrieving data from the queue, respectively. If each data item is assigned with a sequence number, then items with larger sequence numbers are suspended from being added—even if they are ready earlier than others. We refer the first data item as the earliest unconsumed element in the queue. Figure 1 depicts the producer-consumer model vividly.

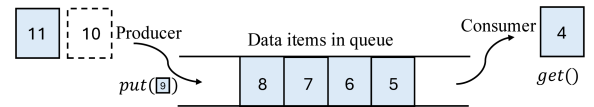


Fig. 1: The producer-consumer model

C. Reliable Broadcast

Reliable broadcast (RBC) ensures that a message broadcast by an honest dealer is received by all non-faulty parties in a distributed system. Particularly, Bracha RBC [28] is regarded as a one-shot consensus algorithm in an asynchronous setting, tolerating $f < n/3$ faulty parties, where n is the number of total parties. Suppose the proposal is v . We slightly modify the Bracha RBC protocol by replacing the proposed value v (of

size $O(n)$) with its hash value hv . The details of the modified protocol are depicted as below:

Bracha RBC for $O(n)$ -size v

- Step 0.** For the dealer j
broadcasts (initial, v).
- Step 1.** For party i , waits until the receipt of
(initial, v)
broadcasts (echo, hv).
- Step 2.** For party i , waits until the receipt of
 $2f + 1$ (echo, hv) or $(f + 1)$ (ready, hv)
broadcasts (ready, hv).
- Step 3.** For party i , accepts v and hv until the
receipt of $2f + 1$ (ready, hv).

Note that if party i receives $2f + 1$ (ready, hv) without v in **Step 3**, it can fetch v from any other parties. The RBC protocol has the following security properties [47], [48]:

- **Validity** If the dealer is non-faulty, then all non-faulty parties eventually delivers the dealer's proposal v .
- **Agreement** If a non-faulty party delivers a value v' , and another non-faulty party delivers v'' , then $v' = v''$.
- **Totality** If a non-faulty party delivers a value, then all non-faulty parties eventually deliver a value.

It is well-known that RBC protocols cannot handle cases with a faulty dealer [26], as it may either prevent consensus entirely or lead to unbounded termination time in asynchronous settings. We define a symbol " $\perp$ ", representing an initial consensus status. Also, $\perp$ can be initialized as the default consensus value for all parties. It can be inferred that consensus value can be changed from $\perp$ to some value $v (\neq \perp)$. However, v will be immutable once upon agreement by all non-faulty parties due to **validity**, **agreement** and **totality**. Therefore, Claim 1 holds.

Claim 1: In the Bracha asynchronous RBC protocol, all non-faulty parties eventually share the same consensus value, either $\perp$ or $v (\neq \perp)$.

D. t -Validated Asynchronous Byzantine Agreement (t -VABA)

Suppose there are a group of n distributed parties and at most $f < n/3$ of them are faulty. In validated asynchronous Byzantine agreement (VABA) protocols [48], [52], each party has an initial value at the beginning and all non-faulty parties reach agreement on a value in the end. Typically, VABA protocols require that the agreement value is a proposal from a party. In this section, we introduce a t -validated asynchronous Byzantine agreement (t -VABA) protocol.

Definition 1 (t -VABA): Each party i has an initial valid value C_i and all parties' goal is to reach agreement on a valid value v (i.e., $\text{Predicate}(v) = \text{true}$). v is none of the initial values, but determined by any t honest parties, where t is a fixed threshold value.

The predicates Predicate is utilized to validate the output common value. Similarly to previous VABA protocols [48], [49], t -VABA is required to achieve the following properties:

- **External-validity:** If an honest party decides on a value v , then v is calculated with any t honest parties' initial values $\{C_i\}$ and $\text{Predicate}(v) = \text{true}$.
- **Agreement:** All non-faulty parties that terminate decide on the same value v .
- **Termination:** If each non-faulty party has a valid initial value C_i , then all non-faulty parties decide on a value v .

III. SYSTEM AND SECURITY MODELS

A. System Model and Assumption

The proposed distributed randomness beacon AsyRand assumes pairwise connected network for broadcast in an open environment. By "broadcast", we mean the message is sent to all parties (including the sender) through the network. The network model is asynchronous, so messages can be arbitrarily delayed. PKI is also assumed so that each party leverages a public key as its identity.

Denote n as the number of parties remaining in the system. f represents the number of faulty parties that may cause Byzantine failures or disobey the protocols. For simplicity, we assume $n = 3f + 1$. Parties can quietly leave without sending any messages. As we allow for parties changing, we assume mobile adversary [50], where the condition $f < n/3$ is assumed to hold regardless of how n varies.

We set $t = f + 1$ for t -VABA protocols. An epoch is the period of consuming a PVSS commitment (via a t -VABA protocol) in the consumer process. Different parties may in different epochs at a specific time due to asynchronous network. The initial beacon value and the initial leader are assumed to be obtained in a decentralized way, which can be obtained with VDF [38] or a nonce from Bitcoin [11].

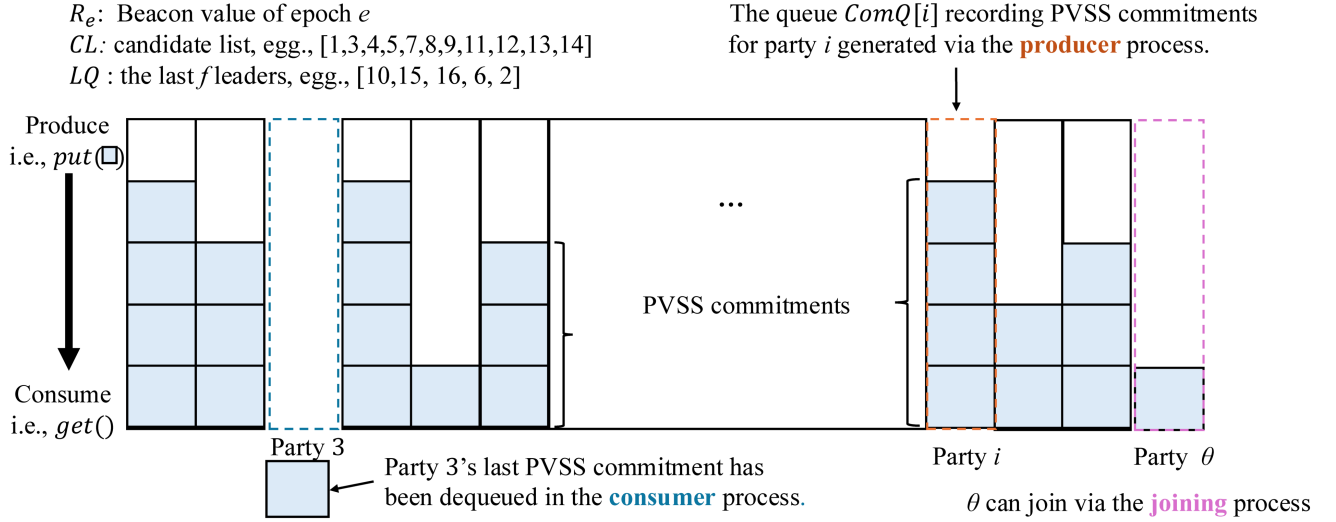
B. Threat Model

In the system, malicious parties or adversaries can collude to violate the above properties by arbitrarily biasing the protocol. A summary of risks and potential attacks is introduced below: In the producer process, each malicious party has the following options: ① delay the delivery of valid PVSS commitment; ② send invalid PVSS commitment to honest parties; ③ send contradicting valid PVSS commitments to different honest parties; ④ send nothing. In the consumer process, a malicious party i also has similar options in broadcasting a PVSS decryption key D_i in each epoch. Besides, the f malicious parties may collude to predict the random beacon values in advance. We consider only one party joining or leaving at a time, as simultaneous reconfigurations can be reduced to a sequence of individual, one-by-one reconfigurations.

IV. THE AsyRand BEACON PROTOCOL

A. High-level Overview and Global States

Figure 2 depicts the high-level overview of AsyRand, depicted using a producer-consumer model.



Party 3 will be removed in the **removal** process if $|ComQ[3]|=0$ holds long, as the condition halts the **consumer** process.

Fig. 2: Local view at a party, depicting the four processes and the queues recording PVSS commitments

In the producer process, each party broadcasts new PVSS commitments, which are the outputs of the PVSS.Share algorithm, via continuously invoking RBC protocols. In the consumer process, a leader is randomly selected for each epoch, and the earliest PVSS commitment produced by this leader is consumed to recover a random value with the t -VABA protocol. Further, the recovered random value is adopted to generate a fresh beacon value.

TABLE II: The global states shared by all processes

State	description
$\mathcal{P}$	the set of all parties
n, f	the number of parties and faulty parties
e	current epoch
L	the leader of current epoch e
R_{e-1}, R_e	beacon value of epoch $e-1, e$
pk_i	the i th party's public key
LQ	a queue recording past f leaders
CL	the leader candidates list, i.e., $CL = \mathcal{P} - LQ$
$ComQ$	collection of FIFO queues
$ComQ[i]$	the FIFO queue recording unconsumed PVSS commitments from party i
seq_i	the sequence number of a PVSS commitment at party i
$queLen$	the maximum PVSS commitments each queue can store

Table II summarize the global states, which are shared among all processes in a party. The leader queue (LQ) records past f leaders. The candidate list (CL) denotes potential leaders, i.e., the parties that have not been leaders in the past f epochs. This design helps avoid the situation where malicious parties dominate leadership consecutively, giving them an undue advantage in the protocol. It is apparent that $CL = \mathcal{P} - LQ$. $ComQ$ is a collection of FIFO queues. Each PVSS commitment from party i carries a monotonically increasing sequence number j , denoted as seq_{ij} . For notational simplicity, we use seq_i throughout this paper rather than seq_{ij} , as the additional index j serves no meaningful purpose in our writing. Thus, $ComQ[i]$ is a queue, with length $queLen$,

recording the PVSS commitments of party i in chronological order. Hence, all parties share the same global states, including $\{pk_i\}, LQ, CL, R_{e-1}, e, L, ComQ, seq_i, queLen$.

B. The Producer Process

In the producer process, each party i independently and continuously invokes RBC protocol to broadcast new PVSS commitments as long as his queue is not full, as depicted by Figure 3. Each of the PVSS commitment is generated using the PVSS scheme inputting a fresh random value $s \in \mathbb{Z}_p$. Particularly, the proposal is set as $v = (i, C, \pi, seq_i)$ in **Step 0**, where (C, π) is output of the PVSS.Share($s, \{pk_i\}$) algorithm and seq_i denotes the seq_i -th PVSS commitment. The public verifiability property of PVSS scheme enables anyone to check whether an RBC dealer has honestly produced PVSS commitments by leveraging the PVSS.Verify algorithm in **Step 1**. Once an RBC instance is accomplished in **Step 3**, each party updates the global state by $ComQ[v.i].put(v)$.

Claim 2 ($ComQ[i]$ is in order): The values in $ComQ[i]_{\forall i}$ are in the right order for all honest parties.

Proof. We leverage seq_i to mark the seq_i -th PVSS commitment of party i . Therefore, it is impossible for an honest party to accept two contradicting PVSS commitments for a sequence number seq_i , due to the RBC **agreement** property. So, values in $ComQ[i]_{\forall i}$ are in the right order for all honest parties.

In an asynchronous RBC protocol, the absence of network timeout assumptions prevents honest parties from reliably determining whether a dealer i is faulty or not. By Claim 1, all non-faulty parties will eventually have the same consensus value $\perp$ or v . If they agree on v , they will append v to $ComQ[i]$ as the seq_i -th item; otherwise, they will take no action to the queue $ComQ[i]$. Thus, honest parties can shift their focus to the status of the queue $ComQ[i]$ and Claim 3 holds.

Claim 3 (Focus on queues): Honest parties in the producer can just focus on the queue status $ComQ[i]$ when party i is the dealer, instead of accessing i 's honesty through message

Step 0. For party i , waits until $|ComQ[i]| \neq queLen$, invokes $(C, \pi) \leftarrow PVSS.Share$. Denotes $v = (i, C, \pi, seq_i)$ and broadcasts (initial, v).

Step 1. For party i , waits until the receipt of (initial, v), where $PVSS.Verify(v.C, v.\pi)$ is true broadcasts (echo, hv), where $hv = Hash(v)$.

Step 2. For party i , waits until the receipt of $2f + 1$ (echo, hv) or $f + 1$ (ready, hv) broadcasts (ready, hv).

Step 3. For party i , waits until the receipt of $2f + 1$ (ready, hv) $ComQ[v.i].put(v)$.

Fig. 3: The producer process at each party i

analysis. Thus, the potential threats in the producer process defined in system model (cf. Section III-B) are addressed.

We prove the safety and liveness for the producer process in an asynchronous setting by Theorem 1 and Theorem 2.

Theorem 1 (Safety of the producer process): No matter whether a dealer i is faulty or not, the result of the producer process, $ComQ[i]$, will reach agreement for all honest parties. *Proof.* The producer process consists of infinite RBC instances invoked by each party. Any two RBC instances led by two dealers are independent. Hence, we discuss the case where any party i invokes the RBC protocols as the dealer. By Claim 3, honest parties can ignore whether a dealer i is honest or not. Moreover, the elements in $ComQ[i]$ are in right order by Claim 2. Therefore, the global state $ComQ[i]$ will be eventually under consensus for all honest parties.

Theorem 2 (Liveness of the producer process): No adversary could prevent the producer process from appending valid PVSS commitment into $ComQ[i]$ for honest party i .

Proof. The producer process consists of infinite independent RBC protocols led by all parties separately. The proposal (i.e., PVSS commitment) of party i does not interfere with the proposal of another party j , since they have independent storage space, i.e., $ComQ[i]$ and $ComQ[j]$. Since $n = 3f + 1$, the $2f + 1$ honest parties continuously propose new valid PVSS commitments, which will reach agreement among them due to RBC **validity**, guaranteeing the liveness of the producers.

C. The Consumer Process

The consumer process continuously choose a random leader L for each epoch e . In each epoch, all parties cooperate to recover L 's first PVSS commitment and reach consensus on a beacon value. Figure 4 depicts the consumer process at party i for some epoch e using a t -VABA protocol (cf. Definition 1). The t -VABA protocol begins with each party holding an initial value C_i (in C) and it reaches agreement on the beacon value R_e in the end. The gray text in Figure 4 considers the situation

whether the leader L is removed or not. The removal process is introduced in Section V-A.

Step 0. For party i , $L = CL[R_{e-1} \bmod |CL|]$ and $v \leftarrow ComQ[L].get()$, parse v as (L, C, π, seq_L) . Waits until $|ComQ[L]| > 0$ or $L.status \equiv removed$:
 $oldL \leftarrow LQ.get()$ and $CL.add(oldL)$
 if $L.status \neq removed$:
 $LQ.put(L)$ and $CL.remove(L)$
 $D_i \leftarrow PVSS.PreRecon(C_i, sk_i)$
 broadcasts (recon, D_i).

Step 1. For party i , waits until the receipt of t (recon, D_i),
 $gs \leftarrow PVSS.Recon(C, \{D_i\})$
 $R_e \leftarrow Hash(R_{e-1}, gs)$
 broadcasts (reconEcho, e, R_e).

Step 2. For party i , waits until the receipt of $2f + 1$ (reconEcho, e, R_e) or $f + 1$ (reconReady, e, R_e) broadcasts (reconReady, e, R_e).

Step 3. For party i , waits until the receipt of $2f + 1$ (reconReady, e, R_e),
 accepts R_e
 enters **Step 0** to the next epoch $e + 1$.

Fig. 4: The consumer process at party i

The t -VABA protocol follows a four-step structure analogous to the RBC protocol, but operates without a dealer. In **Step 0**, each party determines current leader as $L = CL[R_{e-1} \bmod |CL|]$, where R_{e-1} is the beacon value of previous epoch $e - 1$. The party then obtains L 's first PVSS commitment by $v \leftarrow ComQ[L].get()$, and parse v as (L, C, π, seq_L) . The waiting condition $|ComQ[L]| > 0$ means that the t -VABA protocol really starts only if L has 1 or more PVSS commitments to consume. This design proactively prevents the case where L is re-elected as leader without having an unused PVSS commitment available, thus eliminating potential liveness stalls in the consumer process.

If the above condition is met, the oldest leader $oldL$ (i.e., $LQ.get()$) should be placed into candidate list CL . And if L is still alive in the system, it should be transferred from the candidate list CL to the leader queue LQ . The encrypted share C_i (in C) serves as party i 's initial value. Subsequently, each party i obtains the decryption key $D_i \leftarrow PVSS.PreRecon(C_i, sk_i)$ for the commitment v and broadcast it. In **Step 1**, each party recovers the PVSS secret gs with at least t valid decryption keys, and constructs the value R_e . The **Step 2** and **Step 3** are the same with the RBC protocol (cf. Section II-C). In the end, all non-faulty parties accepts R_e as the beacon value of the epoch e .

In summary, the consumer process updates the global variables, i.e., L , CL , LQ , $ComQ$ and outputs new beacon value

R_e for each epoch e . Obviously, the execution of the consumer process relies on the producer process. The consumer process should be “slower” than the producer process, so that the operation $ComQ[L].get()$ always returns a valid PVSS commitment in every epoch. The fact is guaranteed and it is proved by Claim 4.

Claim 4 ($ComQ[i]$ is non-empty): The set $ComQ[i]$ is non-empty, ensuring that the consumer process always has a valid PVSS commitment to consume whenever a party i is chosen as leader.

Proof. By Theorem 2, the queue $ComQ[i]$ can be appended without interruption. The condition, i.e., $|ComQ[i]| > 0$, in **Step 0** guarantees that $ComQ[i]$ is non-empty for all honest parties. If party i stops producing valid PVSS commitments in the producer process, $ComQ[i]$ will stop growing. $ComQ[i]$ will be empty in **Step 0** if i is chosen as leader for $|ComQ[i]|$ times. In this case, the consumer process will be suspended and it will be resumed after removing i by the removal process.

We argue that the t -VABA implementation satisfies the required security properties through Lemma 1, Lemma 2, and Lemma 3, respectively. Further, we prove the consumer process ensures properties of safety and liveness by Theorem 3 and Theorem 4.

Claim 5 (Same beginning): At the beginning of each epoch e with leader L , all honest parties share the same value of $v \leftarrow ComQ[L].get()$ in **Step 0**.

Proof. In **Step 1** of each t -VABA protocol, R_e is deterministic due to correctness of PVSS scheme in computing gs and uniqueness of hash in calculating R_e . Thus, it can be easily proved that honest parties have the same output at the beginning of each epoch by applying mathematical induction. Moreover, the producer process guarantees that all honest parties have correct order of $ComQ[L]$ (cf. Claim 2). And there is at least one PVSS commitment in the queue $ComQ[L]$ (cf. Claim 4). Therefore, all honest parties have the same value of $ComQ[L].get()$ at the beginning of each epoch.

Lemma 1 (External-validity of each epoch): If an honest party outputs a beacon value R_e for epoch e , then R_e is calculated with any t honest initial values $\{C_i\}$ and $\text{Predicate}(R_e) = \text{true}$.

Proof. By Claim 5, honest parties have the same PVSS commitment to consume at each epoch. Specifically, the initial value for each party is $C_i \leftarrow v.C_i$, where $v \leftarrow ComQ[L].get()$ and it is parsed as $(L, C = \{C_i\}, \pi, seq_L)$. Moreover, C_i has been validated by the PVSS.Verify algorithm in the producer process. Then, each honest party i will decrypt C_i to obtain D_i via PVSS.PreRecon and broadcast (recon, D_i) in **Step 0**. Each party can collect at least $n - f > t$ valid $\{D_i\}$, enabling it to calculate gs by PVSS.Recon($C, \{D_i\}$). Further, the beacon value R_e is calculated as $R_e = \text{Hash}(R_{e-1}, gs)$. All non-faulty parties will send $(\text{reconEcho}, e, R_e)$ in **Step 1**. The subsequent steps (i.e., **Step 2** and **Step 3**) are the same with RBC protocol, guaranteeing that all honest parties will receive $2f + 1$ reconReady messages and accept the same beacon value R_e as the beacon value for epoch e . Therefore, the Predicate roughly consists of PVSS.PreRecon, PVSS.Recon and Hash algorithms.

Claim 6: No two honest parties will send conflicting messages $(\text{reconReady}, e, R_e)$ and $(\text{reconReady}, e, R'_e \neq R_e)$, given the correctly chosen leader L at epoch e .

Proof. Claim 5 has demonstrated that honest parties are consuming the same commitment for epoch e . Suppose two honest parties i and j send reconReady message for two different beacon values R_e and R'_e , respectively. Party i must have received a set A of $2f + 1$ reconEcho for R_e and party j must have received a set B of $2f + 1$ reconEcho for R'_e . Since $|A| = |B| = 2f + 1$, then $|A \cap B| \geq f + 1$ due to quorum intersection property. This implies that at least $f + 1$ parties have sent an echo to both i and j . It means that at least one honest party must have sent two reconReady messages for different values, violating the assumption.

Lemma 2 (Agreement of each epoch): If a non-faulty party accepts R_e , then all non-faulty parties accept R_e for epoch e .

Proof. First consider the scenario where faulty parties collude or send invalid decryption keys $\{D_i\}$ for current leader's PVSS commitment in **Step 0**. However, invalid keys can be detected by the PVSS.Recon($C, \{D_i\}$) algorithm and they will be abandoned. And the PVSS threshold $t = f + 1 > f$, making it impossible for colluding parties to recover the leader L 's PVSS secret gs . Further, Claim 6 shows that no contradicting beacon values can be output for two honest parties. Hence, all honest parties will reach agreement on the same beacon value R_e for epoch e .

Lemma 3 (Termination of each epoch): The consumer process eventually outputs a valid beacon value R_e for the epoch e .

Proof. By Claim 4, $ComQ[i]$ is always non-empty for each epoch. When a party i is elected as leader, **Step 0** can be guaranteed to be executed for all honest parties. And honest parties have the same PVSS commitment to consume by Claim 5. The subsequent steps **Step 1-3** follow the same structure as the RBC protocol. Thus, all honest parties will eventually accept R_e for epoch e , similar to RBC validity. Hence, each epoch achieves the **termination** property of t -VABA protocol.

Theorem 3 (Safety of the consumer process): All non-faulty parties output the same beacon value in each epoch.

Proof. The safety property is implied by the **agreement** of each epoch (cf. Lemma 2). Thus, the consumer process will always reach agreement for all honest parties for any epoch.

Theorem 4 (Liveness of the consumer process): No adversaries could prevent the consumer process from outputting a new beacon value and forwarding to the next epoch.

Proof. The t -VABA **termination** property implies that each epoch eventually ends. Besides, the consumer process can successfully move to the next epoch due to Claim 4. Thus, the consumer process normally proceeds with infinite t -VABA instances. We further consider an abnormal case, where the **Step 0** may suspend due to $|ComQ[L]| = 0$ holds. However, in this occasion, party L will be removed in the removal process and it will not be elected as leader. The removal process notifies the consumer process to go on from the suspended point in **Step 0**. Therefore, the consumer process is guaranteed to achieve liveness.

V. RECONFIGURATION

A. The Removal Process

The removal process is designed to enable honest parties to automatically remove faulty members without system restart. Implied by Claim 3, we innovatively invent a mechanism to discover faulty parties in an asynchronous network. The mechanism allows a party to simply detect whether another party is faulty/malicious by monitoring queue length of the party. If the queue is empty for a long time, honest parties can start to remove the party using a RBC-like protocol, which runs without proposer.

For example, a party L may behave maliciously in the producer process, and no PVSS commitments will be added into $ComQ[L]$ for all honest parties. $ComQ[L]$ will eventually become empty after L is elected as leader for $|ComQ[L]|$ times. Figure 5 depicts the party removal process in a RBC-like protocol.

- Step 0.** For party i , if $|ComQ[L]| = 0$ holds for an expiration event. Denote $v = (L, e)$, broadcasts $(\text{removal}, v)$.
- Step 1.** For party i , waits until the receipt of $2f + 1$ $(\text{removal}, v)$, broadcasts $(\text{removalEcho}, v)$.
- Step 2.** For party i , waits until the receipt of $2f + 1$ $(\text{removalEcho}, v)$ or $f + 1$ $(\text{removalReady}, v)$, broadcasts $(\text{removalReady}, v)$.
- Step 3.** For party i , waits until the receipt of $2f + 1$ $(\text{removalReady}, v)$, set $L.\text{status} = \text{removed}$, notify the consumer process, rollback to epoch $v.e$, if needed.

Fig. 5: The removal process to remove L at party i

If current leader L has no unconsumed commitments, i.e., $|ComQ[L]| = 0$ in the consumer process until an expiration event¹, honest party i broadcasts the initialization message $(\text{removal}, v)$, where $v = (L, e)$. Once a party i receives $2f + 1$ $(\text{removalReady}, v)$, it marks that L is removed and informs its consumer process, which was suspended in **Step 0** in consumer process.

Different parties are chosen as leaders in consecutive f epochs, so multiple removals of different parties are independent. Therefore, we only consider the one-by-one removal in this paper.

¹In our design, the expiration event is defined as the condition where at least $2f + 1$ queues contain PVSS commitments. This threshold aligns with the mobile adversary model's core assumption—that at least two-thirds of the remaining participants are honest.

We prove the removal process satisfy the required **validity**, **agreement** and **termination** properties of RBC-like protocol by Lemma 4, Lemma 5 and Lemma 6, respectively.

Lemma 4 (Validity of the removal process): If at least $2f + 1$ non-faulty parties broadcast **removal** for a leader in epoch e , then all non-faulty parties will remove the leader.

Proof. If at least $2f + 1$ non-faulty parties broadcast **removal**, then all of them will broadcast **removalEcho** and **removalReady** in **Step 1** and **Step 2**, respectively. Then all non-faulty parties will remove the leader upon receiving $2f + 1$ **removalReady**.

Claim 7: No two honest parties will send conflicting message $(\text{removalReady}, v)$ and $(\text{removalReady}, v' \neq v)$, where $v.e = v'.e$.

Proof. That means given a specific epoch e , no removal agreement will achieve to remove multiple leaders. The proof follows the same reasoning as in Claim 6 and is therefore omitted here.

Lemma 5 (Agreement of the removal process): If a non-faulty party agrees to remove party L who is chosen as leader in epoch e , then eventually all non-faulty parties will agree to remove L .

Proof. If a non-faulty party agrees to remove party L , it must have received $2f + 1$ $(\text{removalReady}, v)$ messages, of which at least $f + 1$ came from non-faulty parties. Consequently, all non-faulty parties will broadcast $(\text{removalReady}, v)$, either due to seeing the $f + 1$ **removalReady** messages or due to seeing $2f + 1$ echoes, as described in Figure 5 **Step 2**. Claim 7 guarantees that no non-faulty will broadcast $(\text{removalReady}, v' \neq v)$. So there will not be $2f + 1$ echoes for v' or $f + 1$ **removalReady** for v' . Hence, eventually all non-faulty parties will agree to remove party L .

Lemma 6 (Termination of the removal process): All non-faulty parties will eventually terminate with agreement on whether L is removed or not.

Proof. If L stops providing valid PVSS commitments and it will be removed by honest parties. Here, we consider the worst-case scenario, where a fork may occur among honest parties. Define a set A in which at least one honest party has removed L , another set B containing the other honest parties who have not removed L yet. Due to **agreement** of the removal process (cf. Theorem 5), all parties in set B will eventually switch to set A at epoch e^* . e^* is also called the termination epoch, which is determined by the $(|B| - f)$ -th party in set B . Since with only f honest parties and f correct parties, no new beacon values can be delivered.

By Claim 8, we prove safety of the removal process even though $e^* > e$.

Claim 8: If a fork arises due to the removal process, it will be eventually resolved.

Proof. Suppose e^* to be the determination epoch of the removal process, as defined in Lemma 6. During the period $e^* - e$, at most $e^* - e$ PVSS commitments are consumed and $e^* - e$ beacon values will have been generated. Since the status of parties in the forking set B will eventually rollback, the fork and the $e^* - e$ associating beacon values will be abandoned. This implies that unpredictability might

be affected if $e^* - e > 0$.² Summarily, all honest parties will continue the beacon protocol without party L starting from epoch e and share the same global state $(ComQ, CL, LQ, R_e)$, even though some parties rely on rollback.

B. The Joining Process

We further introduce a party joining process that enables the seamless addition of a new party without system restart. The joining process is described in an RBC protocol execution.

Suppose a joining party θ composes a proposal $v = (e, e^*, pk_\theta, C, \pi, seq_\theta = 1)$, where e is the current epoch, $e^* \gg e$ is the expected epoch for θ to appear in the system, pk_θ is θ 's public key, (C, π) is the output of $PVSS.Share(s, \{pk_i\} \cup pk_\theta)$ and seq_θ is initialized to be 1 as the first proposal. Then, θ initiates the RBC protocol to request to join the system by broadcasting $(join, v)$. Denote the actual termination epoch as $e^\#$. Honest parties can refuse other joining requests within before e^* , thus, we can only consider the one-by-one joining in this paper. Our work does not analyze the placeholder attack—an attack scenario wherein a joining party deliberately blocks other nodes from joining before reaching e^* .

Figure 6 depicts the party joining process using RBC protocol. The actual agreement time may be beyond the expected time, i.e., $e^\# > e^*$. Thus, in **Step 3**, a party may need to rollback status of $(ComQ, CL, LQ, R_{e^*})$ to the expected epoch e^* . We consider the worst situation where rollback occurs by Claim 9.

Step 0. For a joining party θ , invokes $(C, \pi) \leftarrow PVSS.Share$. Set $v = (e, e^*, pk_\theta, C, \pi, seq_\theta = 1)$ and broadcasts $(join, v)$.

Step 1. For party i , waits until the receipt of $(join, v)$, where $PVSS.Verify(v.C, v.\pi)$ is true broadcasts $(joinEcho, hv)$, where hv is hash of v .

Step 2. For party i , waits until the receipt of $2f + 1$ $(joinEcho, hv)$ or $f + 1$ $(joinReady, hv)$ broadcasts $(joinReady, hv)$.

Step 3. For party i , waits until the receipt of $2f + 1$ $(joinReady, hv)$, and wait until epoch e^* or current epoch $e^\# > e^*$,
if $e^\# > e^*$:
rollback $(ComQ, CL, LQ, R_{e^*})$ to epoch e^*
 $ComQ[\theta].put(v)$
 $CL.add(\theta)$

Fig. 6: The joining process for party θ at party i

Claim 9: If an honesty party j accepts θ in **Step 3** at the epoch $e^\# > e^*$. We claim that any fork, if appeared with little probability, will be temporary.

²To resolve the unpredictability problem, we can design a mechanism to make honest parties abandon the $e^* - e$ PVSS commitments and it is not discussed in this paper.

Proof. Define a set A of which honest parties that have accepted θ before the expected epoch e^* , and another set B of honest parties that have not. Obviously, $|A| + |B| = 2f + 1$. ($|A|$ can be 0 in our design.) Due to Claim 1, parties in B will eventually rollback to epoch e^* and align their status with those in A . Summarily, all honest parties eliminate the fork and share the same state in epoch e^* , though some honest parties rely on rollback. Starting from epoch e^* , newly generated PVSS commitments in the producer process will include the public key of θ . Further, we discuss the impact on unpredictability of the beacon. We consider that party j is changed from B to A . Denote $e^\# > e^*$ as the termination epoch, determined by the last party in B . Hence, at most $e^\# - e^*$ beacon values may have been generated with $e^\# - e^*$ PVSS commitments being decrypted in the consumer process. This implies that unpredictability may be compromised for a maximum of $e^\# - e^*$ epochs in the worst case.

VI. PROPERTIES OF AsyRand

Theorem 5 (Liveness/Availability): No adversaries could prevent the processes in AsyRand from proceeding.

Proof. We discuss all the four processes in AsyRand described in Section IV. By Theorem 2, we prove that no adversaries can prevent new PVSS commitments from being appended into global state $ComQ$ for honesty parties. By Theorem 4, we prove that no adversaries could prevent a PVSS commitment from being consumed. Party L can be removed in the removal process by all honest parties, if $ComQ[L]$ remains empty for a long time. A new party can request to join in the system with an RBC protocol. Both the removal and joining processes eventually reach agreement due to Lemma 6 and RBC **validity**, respectively. Thus, adversaries cannot prevent the processes of AsyRand from proceeding within the proposed producer-consumer model, ensuring liveness. Further, the PVSS commitment queue $ComQ[L]$ of the leader L is always non-empty during the consumer process by Claim 4. Then, a secret gs hidden in $v \leftarrow ComQ[L].get()$ can be eventually recovered with $t (= f + 1)$ PVSS decrypted shares from honest parties. Therefore, a new beacon value R_e for epoch e is guaranteed to be calculated by $Hash(gs, R_{e-1})$, showing property of guaranteed output delivery [12], [10].

Theorem 6 (Bias Resistance): Adversaries cannot bias the beacon output R_e of epoch e in a predictable way.

Proof. As designed, beacon values are delivered in the consumer process. In the consumer process, the condition $|ComQ[L]| > 0$ holds (cf. Figure 4) for leader L when the earliest PVSS commitment $v \leftarrow ComQ[L].get()$ is consumed. Moreover, the leader queue LQ guarantees that a party can be chosen as leader again only after f epochs. Hence, the random beacon value $R_e = Hash(gs, R_{e-1})$ depends on a previously committed secret gs and last beacon value R_{e-1} . gs and R_{e-1} are available in the epoch e and at the end of epoch $e - 1$, respectively. Summarily, R_e cannot be controlled or predicted by adversaries in the current epoch e . Thus, it is impossible for adversaries to bias the beacon output R_e in a meaningful manner.

Theorem 7 (Unpredictability): An adversary should not be able to predict (precompute) a future beacon value R_{e+f+1} , where e is the epoch of current time.

Proof. As described in Theorem 6, predicting a future random beacon value R_e requires knowledge of both PVSS secret gs and the previous beacon value R_{e-1} . PVSS **INDI-Secrecy** ensures that the PVSS secret remains indistinguishable until it is reconstructed. The PVSS secret can be recovered only when at least $t(> f)$ distinct (recon, D_i) messages are broadcast. Consider the worst-case scenario where the f colluding malicious parties share their PVSS secrets privately in real time. If $l \leq f$ of these malicious parties are selected as leaders in consecutive epochs starting from epoch e , then the beacon values from epoch e to epoch $e+l$ can be calculated in advance. The probability of this case can be modeled as hypergeometric distribution [12]. A party will be elected as leader at least f epochs later, making it impossible to predict beacon value after epoch R_{e+f+1} .

Theorem 8 (Public Verifiability): Any third party with publicly known information can verify the correctness of beacon value R_e of each epoch e .

Proof. The public verifiability property of AsyRand is fundamentally inherited from the underlying PVSS scheme. In the producer process, PVSS commitments are generated using the PVSS.Share algorithm and broadcast in an open network, making them publicly verifiable with the PVSS.Verify algorithm. In the consumer process, the decrypted PVSS shares, obtained using the PVSS.PreRecon algorithm, are also broadcast in open network. These shares are then further verified with the PVSS.Recon algorithm before being used to recover the PVSS secret. Both the PVSS algorithms and the calculation of beacon value are deterministic, enabling any third party to publicly verify the entire process of generating R_e .

Theorem 9 (Responsiveness): AsyRand is responsive, meaning that beacon values are delivered at the speed of the real network.

Proof. By Claim 4, there always exists a PVSS commitment to consume in each epoch. Thus, beacon values can be delivered as the consumer process proceeds. From a resource competition perspective, the consumer process is typically slower than the producer. This is because while RBC instances (in the producer) run in parallel across all parties, t -VABA execution (in the consumer) requires joint participation. Theoretically, n PVSS commitments can reach consensus within the time needed to output one beacon value. To achieve this balance, our implementation strategically throttles the producer to optimize consumer throughput, with *queLen* designed as the maximum queue length. Therefore, AsyRand delivers beacon values at the pace of the consumer process, advancing at the speed of the actual asynchronous network.

VII. A EFFICIENT PVSS IMPLEMENTATION

A. Intuition and Construction

Previous studies [23], [39], [15] have shown that Shamir secret sharing (cf. Section B) is a primary ingredient to implement PVSS schemes. To ensure that the secret shares can be publicly transferred, each share $p(i)$ should be properly

encrypted, where $p(x)$ is a polynomial and $s = p(0)$ is the secret. A natural intuition is to hide the i th share with the i th shareholder's public key pk_i , i.e., $pk_i^{p(i)}$. The dealer should then prove honesty in the hiding, enabling the encrypted shares to be publicly verifiable.

Sigma protocol is a practical approach to generate zero knowledge proofs, which can be made non-interactive using the Fiat-Shamir heuristic (cf. Section A). Thus, the dealer can prove knowledge of witness $p(i)$ by publishing the statement $pk_i^{p(i)}$, leveraging NIZK proofs from the Sigma protocol. Particularly, the dealer generates NIZK proofs for each statement $\{pk_i^{p(i)}\}$ for all $i \in [1, n]$. Moreover, the NIZK proofs should also enable to prove that the n witnesses $\{p(i)\}_{i \in [1, \dots, n]}$ correspond to the unique secret s . In our solution, we establish this correspondence by leveraging Lagrange interpolation.

Figure 7 depicts the concrete construction of the proposed PVSS scheme using Sigma protocol and NIZK proof.

Functionality The proposed PVSS scheme

$(\{sk_i, pk_i\}) \leftarrow \text{PVSS.Setup}(\lambda, t, n) :$
 $g \in \mathbb{G}, sk_i \xleftarrow{R} \mathbb{Z}_p, pk_i \leftarrow g^{sk_i}$
 $(C, \pi) \leftarrow \text{PVSS.Share}(s, \{pk_i\}) :$
 $p(x) \xleftarrow{R} \text{poly}(\cdot), \text{ where } p(0) = s$
 $C = \{C_i = pk_i^{p(i)}\}_{i \in [1, \dots, n]}$
 $p'(x) \xleftarrow{R} p(x), \text{ where } p'(0) = s' \xleftarrow{R} \mathbb{Z}_p$
 $C' = \{C'_i = pk_i^{p'(i)}\}_{i \in [1, \dots, n]}$
 $\pi \leftarrow \begin{cases} C', \\ c = H(C, C'), \\ \tilde{s} = s' - cs, \\ \{\tilde{p}(i) = p'(i) - c \cdot p(i)\}_{i \in [1, \dots, n]}, \end{cases}$
 $\text{bool} \leftarrow \text{PVSS.Verify}(C, \pi) :$
 $\begin{cases} \{C'_i \stackrel{?}{=} pk_i^{\tilde{p}(i)} \cdot C_i^c\}_{i \in [1, \dots, n]}, \\ \tilde{s} \stackrel{?}{=} \text{intpl}(\{(i, \tilde{p}(i))\}_{i \in [1, \dots, n]}) \end{cases}$
 $D_i \leftarrow \text{PVSS.PreRecon}(C_i, sk_i) :$
 $D_i = C_i^{1/sk_i} = g^{p(i)}$
 $gs \leftarrow \text{PVSS.Recon}(C, \{D_i\}_{i \in T}) :$
 $e(D_i, pk_i) \stackrel{?}{=} e(g, C_i) \forall i \in T$
 $\mu_i = \prod_{j \in T, j \neq i} \frac{j}{j-i}$
 $gs \leftarrow \prod_{i \in T} D_i^{\mu_i} = \prod_{i \in T} g^{\mu_i \cdot p(i)} = g^{\sum_{i \in T} \mu_i \cdot p(i)} = g^s$

Fig. 7: Construction of the proposed PVSS scheme

The PVSS.Setup algorithm takes the security parameter λ , n, t as the input and generates an independent generator g of $\mathbb{G}$. In this algorithm, each shareholder P_i generates a key pair (sk_i, pk_i) and the dealer collects all the public keys $\{pk_i\}$.

In PVSS.Share, the dealer inputs a secret $s \xleftarrow{R} \mathbb{Z}_p$ and all shareholders' public keys $\{pk_i\}$. The dealer chooses a random polynomial $p(x)$ with the condition $p(0) = s$. Calculate each encrypted share $p(i)$ as $C_i = pk_i^{p(i)}$. Denote $C = \{C_i\}_{i \in [1, \dots, n]}$. Further, the dealer generates the Sigma pro-

tol transcript, i.e., the Sigma commitment C' , the challenge c , the response $\tilde{s}$ and $\tilde{p}(i)$. Denote $\pi = (C', c, (\tilde{s}, \{\tilde{p}(i)\}))$ as the NIZK proof. Finally, the dealer publishes the PVSS commitment, i.e., (C, π) publicly.

With PVSS.Verify, anyone can check the validity of the commitment (C, π) . In the algorithm, each encrypted share C_i is verified separately and the Lagrange interpolation is leveraged to verify that all shares are generated from the secret s . Particularly, $\text{intpl}(\{(i, \tilde{p}(i))\}_{i \in [1, \dots, n]})$ recovers the secret of polynomial $\tilde{p}(x)$, i.e., $\tilde{p}(0)$, where $\tilde{p}(x) = p'(x) - c \cdot p(x)$. That is correct due to additive homomorphism of polynomials.

In PVSS.PreRecon, each shareholder P_i privately decrypts the share $D_i = C_i^{1/\text{sk}_i}$ and sends D_i to a recoverer.

In PVSS.Recon, the recoverer firstly checks validity of D_i from each shareholder. Denote T as the indexes of t valid decrypted shares. Then, the recoverer constructs the dealer's secret $gs = g^s$, in which Lagrange interpolation is implied.

The required security properties are proved in Appendix D.

B. Complexity

Computation Complexity: In the PVSS.Share phase, the dealer costs n exponentiations to generate C . And the NIZK proofs, generated from Sigma protocol, $\pi = (C', c, \tilde{s}, \{\tilde{p}_i\}_{i \in n})$, where C' also takes n exponentiations. Hence, the PVSS.Share phase takes $2n$ exponentiations. In the PVSS.Verify phase, it costs 2 exponentiations for verifying each C_i . Therefore, the PVSS.Verify phase takes $2n$ exponentiations. In the PVSS.Recon phase, it costs $2t$ pairings to check validity of $\{D_i\}$ and t exponentiations to calculate gs .

Communication Complexity: In the sharing phase, the dealer publishes the encrypted shares C and the corresponding NIZK proofs $\pi = (C', c, \tilde{s}, \{\tilde{p}_i\}_{i \in n})$. C contains n elements in $\mathbb{G}$, and the NIZK proofs π contain n elements in $\mathbb{G}$ and $n + 2$ elements in $\mathbb{Z}_p$. In the reconstruction phase, the recoverer receives an array $\{D_i\}_{i \in T}$ to decrypt C . Hence, reconstruction phase requires t elements in $\mathbb{G}$ to recover the secret gs .

TABLE III: Computation complexity

Ref.	Sharing		Verification		Reconstruction	
	Exp.		Exp.	Pair	Exp.	Pair
[15] <i>DBS</i>	$2n$		n	$2n$	$t + 1$	$2t + 1$
[15] <i>DDH</i>	$4n$		$5n$	—	$5t + 3$	0
[16] <i>ALBATROSS</i>	$2n$		$2n$	—	$6t$	—
[17] <i>HEPVSS</i>	$7n$		$4n$	—	$3t$	—
[17] <i>DHPVSS</i>	$n(n - t + 2) + 2$		$n(n - t) + 4$	—	$5t$	—
Ours	$2n$		$2n$	—	t	$2t$

TABLE IV: Communication complexity

Ref.	Sharing		Reconstruction	
	$\mathbb{G}$	$\mathbb{Z}_p$	$\mathbb{G}$	$\mathbb{Z}_p$
[15] <i>DBS</i>	$2n$	0	t	0
[15] <i>DDH</i>	$4n$	$n + 1$	$3t$	$t + 1$
[16] <i>ALBATROSS</i>	$2n$	$n + 1$	$5t$	$4t$
[17] <i>HEPVSS</i>	$3n$	$2n$	t	2
[17] <i>DHPVSS</i>	$n + 2$	1	$3t$	t
Ours	$2n$	$n + 2$	t	0

By Table III and Table IV, we compare the complexity between our proposed PVSS scheme and previous schemes [15], [16], [17], where n is the number of shareholders and t is the threshold value.

VIII. IMPLEMENTATION AND EVALUATION

We implement the proposed PVSS scheme with Charm-Crypto library, which is a framework for constructing cryptographic schemes. The Charm-Crypto framework is written in Python language, however it has plausible performance due to reliance of the GMP library [53] and the PBC library [54]. We choose an asymmetric curve *MNT159* to implement the proposed PVSS scheme. We implement a fully-connected peer-to-peer (p2p) network based on TCP socket programming to enable pair-wise communication. Both the consumer and the producer processes share the same P2P network interface, thereby competing for network resources. Our experiments are executed on 128 AWS cloud servers t4g.medium scattered in 8 regions, namely, Canada, Ireland, Ohio, Paris, SaoPaulo, Seoul, Singapore and Sydney. Each of the servers is with 2 vCPUs and 4 GB RAM (similar to SPURT[34]) and runs Linux ubuntu-bionic-18.04 with Python 3.6.9.

TABLE V: Cryptographic cost

$\text{Exp}_{\mathbb{G}_0}$	$\text{Exp}_{\mathbb{G}_1}$	Pair	$ \mathbb{G}_0 / \mathbb{G}_1 $	$ \mathbb{Z}_p $
0.46ms	4ms	3.6ms	100B/304B	48B

We begin by evaluating the cryptographic operation costs of the curve *MNT159*, as shown in Table V. Each element on $\mathbb{G}_0, \mathbb{G}_1, \mathbb{Z}_p$ element costs 100 Bytes, 304 Bytes and 48 Bytes, respectively. It is evident that the bilinear pairing ($e : \mathbb{G}_0 \times \mathbb{G}_1 \rightarrow \mathbb{G}_T$) and $\text{Exp}_{\mathbb{G}_1}$ incur a significantly higher cost than $\text{Exp}_{\mathbb{G}_0}$.³

We then assess the performance of our proposed PVSS scheme in comparison with related PVSS schemes. Figures 8, 9, and 10 illustrate the computation costs for sharing, verification, and reconstruction phases, respectively. Since $\mathbb{G}_1$ (cf. $\text{Exp}_{\mathbb{G}_1}$ in Table V) is used in *SCRAPE_{DBS}* [15], it results in the highest computation overhead in the distribution phase. The results also suggest that *ALBATROSS* [16], which builds upon *SCRAPE_{DDH}* [15], shifts the computational burden from the sharing and verification phases to the reconstruction phase. The superlinear costs in sharing (cf. Figure 8) and verification (cf. Figure 9) phases of *DHPVSS* arise from evaluating a random $(n-t-1)$ -degree polynomial at each i , where $i \in [1, \dots, n]$, leading to approximately $O(n^2)$ exponentiations. *SCRAPE_{DBS}* and our PVSS incur higher computational costs than *HEPVSS*, *DHPVSS* and *SCRAPE_{DDH}* in the reconstruction phase (see Figure 10), primarily because they use bilinear pairings instead of DLEQ for verifying decrypted PVSS shares. Figures 11 and 12 compare the communication overhead during the sharing and reconstruction phases for the dealer and the recoverer as the number of parties n increases. Our PVSS scheme demonstrates competitive performance in terms of communication efficiency.

³In some programming libraries or on certain elliptic curves, such as *SS512*, the bilinear pairing is faster than $\text{Exp}_{\mathbb{G}_0}$.

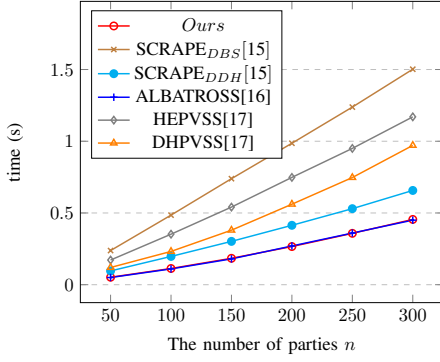


Fig. 8: Comp. cost in the sharing

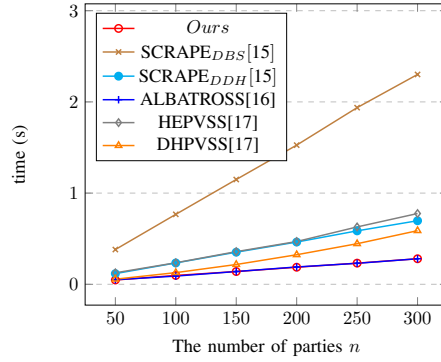


Fig. 9: Comp. cost of in verification

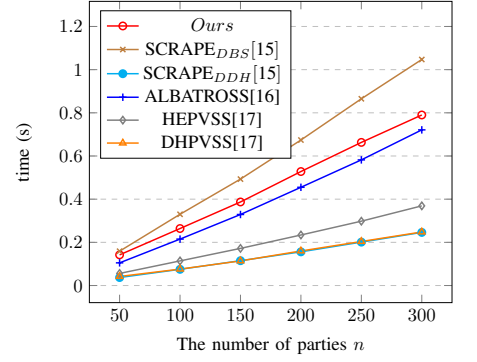


Fig. 10: Comp. cost in reconstruction

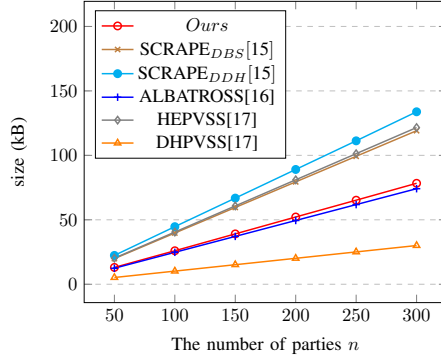


Fig. 11: Comm. cost in the sharing

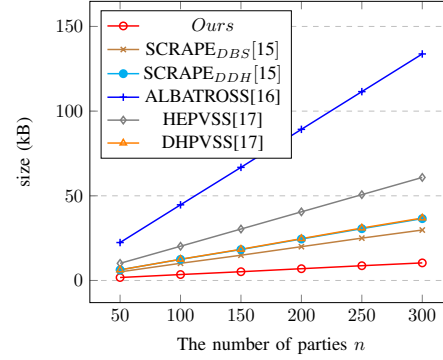


Fig. 12: Comm. cost in reconstruction

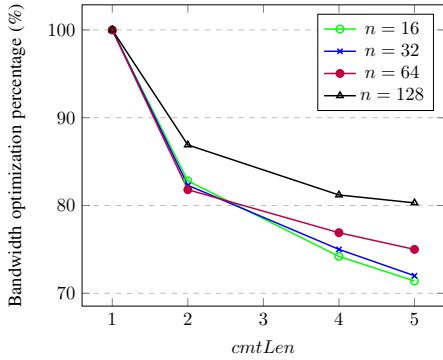


Fig. 13: Bandwidth optimization with multiple PVSS commitments broadcast

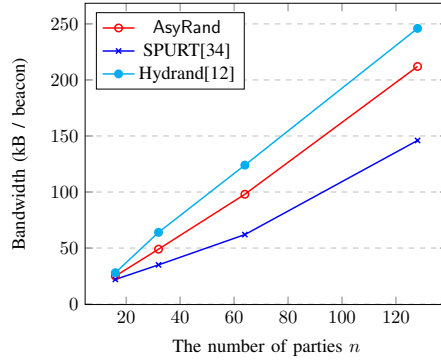


Fig. 14: Bandwidth per beacon

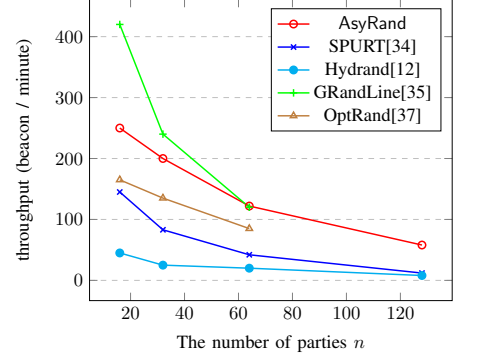


Fig. 15: Average beacon throughput per minute

Next, we evaluate the performance of AsyRand under different configurations with $n = 16, 32, 64, 128$. The producer-consumer model decouples the production and consumption of PVSS commitments, making it convenient to identify performance bottlenecks. As expected, the producer process operates faster than the consumer process. Theorem 9 demonstrates that AsyRand beacon values are generated at the speed of the consumer process. To prevent the consumer process from lagging in computational resource allocation, we introduce parameters to regulate the producer's speed. These parameters include $queLen$ (the maximum queue length) and $cmtLen$ (the number of PVSS commitments sent at a time by a party). Adjusting $queLen$ effectively slows down the producer process, while modifying $cmtLen$ reduces bandwidth consumption, as

multiple PVSS commitments can be transmitted via a single RBC protocol.

By setting $queLen$ to 3, the producer process is slowed with minimal impact on throughput. Adjusting $cmtLen$ to 2, 4, or 5 reduces bandwidth by approximately 17%-27% compared to $cmtLen = 1$. Figure 13 shows the results of bandwidth optimization.

Figure 14 illustrates the bandwidth usage (both sent and received data) per beacon output. For $n = 32$ with $queLen = 2$ and $cmtLen = 4$, the average bandwidth of a party is around 51kB per beacon. The bandwidth usage in AsyRand is higher than in SPURT [34] (35 kB per beacon), because two consensus algorithms are occurring independently in both the producer and consumer processes. However, this increased

bandwidth does not hinder AsyRand’s high throughput in an asynchronous setting.

As illustrated by Figure 15, AsyRand achieves significantly higher throughput compared to Hydrand [12], SPURT [34] and OptRand [37]. Specifically, with $n = 128$, AsyRand produces 58 beacons per minute, while SPURT and Hydrand produce only 12 and 8 beacons per minute, respectively. The throughput of GRandLine [35] exhibits significant degradation as n increases, likely due to its reliance on synchronous network assumptions. In synchronous network, network delay upper bound is usually hard-coded. Though GRandLine has a high throughput when $n < 64$, but it might become insecure if the predetermined time bound is exceeded.

IX. CONCLUSION

In this paper, we introduce AsyRand, a distributed randomness beacon protocol designed for asynchronous network setting. AsyRand operates within a producer-consumer model. Each producer process generates PVSS commitments, which are then consumed by all consumer processes to continuously produce beacon values. To reach consensus among distributed parties, we integrate RBC and t -VABA protocols in producer and consumer, respectively. Additionally, AsyRand supports reconfiguration, enabling the removal of faulty parties and the addition of new parties without system restart. Our analysis establishes that AsyRand achieves key properties, including liveness, unpredictability, bias-resistance, public verifiability and responsiveness. Experimental results further validate the feasibility and effectiveness of AsyRand. As an independent contribution, we introduce a novel PVSS scheme based on Sigma protocol and Fiat-Shamir heuristic. In the future, we will try to address the rollback problem and will explore the producer-consumer model in implementing asynchronous BFT consensus.

REFERENCES

- [1] Rabin, M. O. Transaction protection by beacons. *J. Comput. Syst. Sci.*, 1983, 27(2):256–267.
- [2] Escudero, D., Tjuawinata, I., & Xing, C. On Information-Theoretic Secure Multiparty Computation with Local Repairability. In *PKC*, 2024, pp. 205–239.
- [3] David, B., Magri, B., Matt, C., Nielsen, J. B., & Tschudi, D. Gear-box: Optimal-size shard committees by leveraging the safety-liveness dichotomy. In *CCS*, 2023, pp. 683–696.
- [4] Hu, D., Wang, J., Liu, X., Li, Q., & Li, K. LMChain: An Efficient Load-Migratable Beacon-based Sharding Blockchain System. *IEEE TC*, 2024.
- [5] Hou, R., Yu, H., & Saxena, P. Using throughput-centric byzantine broadcast to tolerate malicious majority in blockchains. In *S&P*, 2022, pp. 1263–1280.
- [6] Kiayias, A., Russell, A., David, B., & Oliynykov, R. Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol. In *CRYPTO*, 2017, pp. 357–388.
- [7] Van Den Hooff, J., Lazar, D., Zaharia, M., & Zeldovich, N. Vuvuzela: Scalable private messaging resistant to traffic analysis. In *POSP*, 2015, pp. 137–152.
- [8] Abraham, I., Chan, T. H., Dolev, D., Nayak, K., Pass, R., Ren, L., & Shi, E. Communication complexity of byzantine agreement, revisited. In *PODC*, 2019, pp. 317–326.
- [9] David, B., Gaži, P., Kiayias, A., & Russell, A. Ouroboros Praos: An Adaptively-Secure, Semi-synchronous Proof-of-Stake Blockchain. In *EUROCRYPT*, 2018, pp. 66–98.
- [10] Choi, K., Manoj, A., & Bonneau, J. SoK: Distributed randomness beacons. In *S&P*, 2023, pp. 75–92.
- [11] Bonneau, J., Clark, J., & Goldfeder, S. On bitcoin as a public randomness source. <http://eprint.iacr.org/2015/1015>, 2015.
- [12] Schindler, P., Judmayer, A., Stifter, N., & Weippl, E. Hydrand: Efficient Continuous Distributed Randomness. In *S&P*, 2020, pp. 73–89.
- [13] Syta, E., Jovanovic, P., Kogias, E. K., Gailly, N., Gasser, L., Khoffi, I., Fischer, M. J., & Ford, B. Scalable Bias-Resistant Distributed Randomness. In *S&P*, 2017, pp. 444–460.
- [14] Bünz, B., Goldfeder, S., & Bonneau, J. Proofs-of-delay and randomness beacons in Ethereum. In *S&B*, 2017.
- [15] Cascudo, I., & David, B. SCRAPE: Scalable Randomness Attested by Public Entities. In *ACNS*, 2017, pp. 537–556.
- [16] Cascudo, I., & David, B. ALBATROSS: Publicly Attestable Batched Randomness Based on Secret Sharing. In *ASIACRYPT*, 2020, pp. 311–341.
- [17] Cascudo, I., David, B., Garms, L., & Konring, A. YOLO YOSO: Fast and Simple Encryption and Secret Sharing in the YOSO Model. In *ASIACRYPT*, 2022, pp. 651–680.
- [18] Gilad, Y., Hemo, R., Micali, S., Vlachos, G., & Zeldovich, N. Algorand: Scaling Byzantine Agreements for Cryptocurrencies. In *SOSP*, 2017, pp. 51–68.
- [19] Hanke, T., Movahedi, M., & Williams, D. Dfinity technology overview series, consensus system. *arXiv preprint arXiv:1805.04548*, 2018.
- [20] Cascudo, I., David, B., Shlomovits, O., & Varlakov, D. M. Random: Multi-tiered randomness beacons. In *ACNS*, 2023, pp. 645–674.
- [21] Das, S., Yurek, T., Xiang, Z., Miller, A., Kokoris-Kogias, L., & Ren, L. Practical Asynchronous Distributed Key Generation. In *S&P*, 2022, pp. 2518–2534.
- [22] Boneh, D., Bonneau, J., Bünz, B., & Fisch, B. Verifiable delay functions. In *Crypto*, 2018, pp. 757–788.
- [23] Schoenmakers, B. A Simple Publicly Verifiable Secret Sharing Scheme and Its Application to Electronic Voting. In *CRYPTO*, 1999, pp. 148–164.
- [24] Zhang, L., Qiu, F., Hao, F., & Kan, H. 1-round distributed key generation with efficient reconstruction using decentralized CP-ABE. *IEEE TIFS*, 2022, 17: 894–907.
- [25] Abraham, I., Bacho, R., Loss, J. and Stern, G. Nearly Quadratic Asynchronous Distributed Key Generation. *Cryptology ePrint Archive*, 2025.
- [26] Castro, M., & Liskov, B. Practical byzantine fault tolerance. In *OSDI*, 1999.
- [27] Yin, M., Malkhi, D., Reiter, M. K., Gueta, G. G., & Abraham, I. HotStuff: BFT consensus with linearity and responsiveness. In *PODC*, 2019, pp. 347–356.
- [28] Bracha, G. An asynchronous $[(n-1)/3]$ -resilient consensus protocol. In *PODC*, 1984, pp. 154–162.
- [29] Damgård, I. On Σ -Protocols, <https://www.cs.au.dk/~ivan/Sigma.pdf>, Accessed: 2025-03-06
- [30] Fiat, A., & Shamir, A. How to prove yourself: Practical solutions to identification and signature problems. In *CRYPTO*, 1986, pp. 186–194.
- [31] Shamir, A. How to share a secret. *Comm. of the ACM*, 1979, 22(11): 612–613.
- [32] Feldman, P. A practical scheme for non-interactive verifiable secret sharing. In *FOCS*, 1987, pp. 427–438.
- [33] Bhat, A., Shrestha, N., Luo, Z., Kate, A., & Nayak, K. Randpiper—reconfiguration-friendly random beacons with quadratic communication. In *CCS*, 2021, pp. 3502–3524.
- [34] Das, S., Krishnan, V., Isaac, I. M., & Ren, L. Spurt: Scalable distributed randomness beacon with transparent setup. In *S&P*, 2022, pp. 2502–2517.
- [35] Bacho, R., Lenzen, C., Loss, J., Ochsenreither, S. and Papachristoudis, D. GRandLine: adaptively secure DKG and randomness beacon with (log-)quadratic communication complexity. In *CCS*, 2024, pp. 941–955.
- [36] Fischer, M. J., Lynch, N. A., & Paterson, M. S. Impossibility of distributed consensus with one faulty process. *JACM*, 1985, 32(2), 374–382.
- [37] Bhat, A., Shrestha, N., Kate, A., & Nayak, K. OptRand: Optimistically Responsive Reconfigurable Distributed Randomness. In *NDSS*, 2023.
- [38] Schindler, P., Judmayer, A., Hittmeir, M., Stifter, N., & Weippl, E. Randrunner: Distributed randomness from trapdoor VDFs with strong uniqueness. In *NDSS*, 2021.
- [39] Heidavand, S., & Villar, J. L. Public verifiability from pairings in secret sharing schemes. In *SAC*, 2009, pp. 294–308.
- [40] Chaum, D., & Pedersen, T. P. Wallet databases with observers. In *CRYPTO*, 1992, pp. 89–105.
- [41] McEliece, R. J., & Sarwate, D. V. On sharing secrets and Reed-Solomon codes. *Comm. of the ACM*, 1981, 24(9), pp. 583–584.
- [42] Stadler, M. Publicly verifiable secret sharing. In *Eurocrypt*, 1996, pp. 190–199.

- [43] Cascudo, I. and David, B. Publicly verifiable secret sharing over class groups and applications to DKG and YOSO. In *Eurocrypt*, 2024, pp. 216-248.
- [44] DLEQ. https://asecuritysite.com/encryption/go_dleq. Accessed: 2025-03-16.
- [45] Meng, X., Sui, X., Yang, Z., Rong, K., Xu, W., Chen, S., Yan, Y. and Duan, S. Rondo: Scalable and Reconfiguration-Friendly Randomness Beacon. In *NDSS*, 2024.
- [46] Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S. and Stern, G., 2022. Bingo: Adaptively Secure Packed Asynchronous Verifiable Secret Sharing and Asynchronous Distributed Key Generation. In *CRYPTO*, 2023.
- [47] Alhaddad, N., Das, S., Duan, S., Ren, L., Varia, M., Xiang, Z., and Zhang, H. Balanced byzantine reliable broadcast with near-optimal communication and improved computation. In *PODC*, 2022, pp. 399-417.
- [48] Guo, B., Lu, Z., Tang, Q., Xu, J., and Zhang, Z. Dumbo: Faster asynchronous bft protocols. In *CCS*, 2020, pp. 803-818.
- [49] Abraham, I., Malkhi, D., and Spiegelman, A. Asymptotically optimal validated asynchronous byzantine agreement. In *PODC*, 2019, pp. 337-346.
- [50] Yung, M. The "Mobile Adversary" Paradigm in Distributed Computation and Systems. In *PODC*, 2015, pp. 171-172.
- [51] Antunes, D.S., Oliveira, A.N., Breda, A., Franco, M.G., Moniz, H. and Rodrigues, R. Alea-BFT: Practical Asynchronous Byzantine Fault Tolerance. In *USENIX NSDI*, 2024, pp. 313-328.
- [52] Lu, Y., Lu, Z., Tang, Q. and Wang, G. Dumbo-mvba: Optimal multi-valued validated asynchronous byzantine agreement, revisited. In *PODC*, 2020, pp. 129-138.
- [53] GMP library, <https://gmplib.org>, Accessed: 2024-07-05.
- [54] PBC library, <https://crypto.stanford.edu/pbc/>, Accessed: 2024-07-05.



Jiheng Zhang received both Ph.D. and M.S. degrees in operations research from Georgia Institute of Technology, MS degree in mathematics from the Ohio State University, and BS degree in mathematics in Nanjing University. He joined Industrial Engineering and Decision Analytics (IEDA) of the Hong Kong University of Science and Technology (HKUST) as an Assistant Professor in 2009, and was promoted to Associate Professor in 2015 and then to full Professor in 2020. He now is the department head of IEDA of HKUST. He is also the associate director of HKUST Crypto-Fintech Lab.



Liang Zhang obtained excellent programming skills in Baidu Co., Ltd. after acquiring bachelor's degree from Huazhong University of Science and Technology. He then acquired Ph.D. degree from Fudan University in 2022. He is currently a postdoctoral fellow at Hong Kong University of Science and Technology. His research interests include blockchain and applied cryptography. His recent publications are included in *IEEE TIFS*, *IEEE TC*, *IEEE TSC*, *IEEE TBD*, *CJ*, *CSCWD'25*, *APWeb-WAIM'23*, *CF'22* and *SACMAT'22*.



Tao Liu was born in 2002. He joined Hainan University in September 2021. He is currently pursuing a bachelor's degree at the School of Cyberspace Security (School of Cryptography) at Hainan University and will pursue a master's degree at the School of Cyber Science and Engineering, Wuhan University, starting from September 2025. His research interests include blockchain and applied cryptography.



Haibin Kan was born in 1971. He received a Ph.D. from Fudan University, Shanghai, China, 1999. From June 2002 to February 2006, he was with the Japan Advanced Institute of Science and Technology as an assistant professor. He went back Fudan University in February 2006, where he is currently a full professor. He is also the Director of the Shanghai Blockchain Engineering Research Center. His research interests include coding theory, cryptography, and computation complexity.

APPENDIX

A. Sigma Protocol and NIZK Proof

In a Sigma protocol [29], a prover ($\mathcal{P}$) demonstrates the validity of a statement x such that a verifier ($\mathcal{V}$) learns nothing about the witness w , where $(x, w) \in R$ represents a relation. A Sigma protocol follows a three-move interaction pattern between the prover and the verifier:

- 1) *Commitment* (a): $\mathcal{P}$ selects a random value r and computes a commitment a . $\mathcal{P}$ sends the a to $\mathcal{V}$.
- 2) *Challenge* (e): $\mathcal{V}$ selects a random challenge e from a challenge space and sends it to $\mathcal{P}$.
- 3) *Response* (z): $\mathcal{P}$ computes a response z using r , e , and the witness w . $\mathcal{P}$ sends the response z to $\mathcal{V}$. Then, $\mathcal{V}$ checks a verification equation involving a , e , and z .

In practice, NIZK proofs can be derived from the Sigma protocol by leveraging Fiat-Shamir heuristic [30], where the challenge value e is calculated by $\mathcal{P}$ with random oracle. Sigma protocols have the following properties:

- **Completeness.** If $\mathcal{P}$ knows the witness, $\mathcal{P}$ can always prove it.
- **Soundness.** If $\mathcal{P}$ does not know the witness, $\mathcal{P}$ cannot convince the verifier that it does.
- **Zero-Knowledge.** Zero-knowledge ensures that the verifier learns nothing beyond the validity of the statement.

B. Shamir secret sharing

Shamir's Secret Sharing [31] allows a dealer to distribute a secret among a group of parties. The underlying principle of Shamir's Secret Sharing is based on polynomial interpolation. Particularly, the dealer chooses a random $(t-1)$ -degree polynomial $p(x) = s + \sum_{j=1}^{t-1} a_j x^j$, where s is the dealer's secret. Then, the shares are calculated as $s_i = p(i)$, for $i \in [1, \dots, n]$. Each share s_i is sent to the corresponding shareholder P_i in a secret channel. Anyone who collects t shares can apply Lagrange interpolation to recover the secret, i.e., $s \leftarrow \text{intpl}(\{(j, \tilde{p}(j))\}_{j \in [i_1, \dots, i_t]})$. Nevertheless, if with less than t shares, no information about the secret is revealed.

C. Summary of the Processes

Bracha Reliable Broadcast (RBC) or t -VABA protocol is a one-shot communication abstraction to achieve consensus. To provide a concise overview of the processes, Table VI is presented. The column **Instance(s)** indicates how many RBC(-like) or t -VABA protocols are executed in each process. The column **Actions** summarizes the PVSS algorithms involved in each process. The column **States** introduces the global states which are updated in each process.

TABLE VI: Summary of the processes in AsyRand

Process	Protocol	Instance(s)	Actions	States
Producer	RBC	infinite	PVSS.Share PVSS.Verify	$ComQ[L]$
Consumer	t -VABA	infinite	PVSS.PreRecon PVSS.Recon	LQ, CL, L $ComQ[L], e, R_e$
Removal	RBC-like	1	—	L
Joining	RBC	1	PVSS.Share PVSS.Verify	$CL, ComQ[\theta]$

The producer and joining processes are directly leverage the RBC protocol. All parties first invoke the PVSS.Share algorithm and initiate the RBC protocol with the output PVSS.commitment as proposal. Subsequently, PVSS.Verify is triggered when a party receives the initial or join message.

The consumer is constructed based on t -VABA protocols. In the consumer process, the PVSS.PreRecon and PVSS.Recon algorithms are executed to recover the leader's secret. Both the producer and consumer processes run with an infinite number of RBC and t -VABA instances, respectively, which form the foundation of AsyRand. The removal and joining processes requires only a single instance of the RBC(-like) protocol.

The RBC protocol has communication complexity of $O(n^2)$ in the producer process, where parties broadcast $O(n)$ -size PVSS commitments. In the consumer process, each party only broadcasts $O(1)$ size messages in each step. Consequently, the complexity of the consumer process is also $O(n^2)$. Therefore, the overall communication complexity of AsyRand is $O(n^2)$.

D. Security Analysis of the Proposed PVSS

By Theorem 10, Theorem 11 and Theorem 12, we prove the required security properties, defined in Section II-A, of the proposed PVSS scheme.

Theorem 10 (Correctness): Given at least t decrypted shares $\{D_i\}_{i \in T}$, the secret gs can be successfully recovered.

Proof. The Lagrange interpolation ensures that $\sum_{i \in T} \mu_i \cdot p(i)$ interpolates to the value $p(0)$, where $\{\mu_i = \prod_{j \in T, j \neq i} \frac{j}{j-i}\}$ are the Lagrange coefficients. Consequently, the PVSS secret gs can be reconstructed as $g^s = g^{p(0)}$ by evaluating $\prod_{i \in T} D_i^{\mu_i}$ where $D_i = g^{p(i)}$. Therefore, the correctness of the PVSS scheme is guaranteed.

Lemma 7 (Completeness of Sigma protocol): If the dealer knows the secret s , he can prove it.

Proof. The dealer, acting as the prover, generates (C, π_s) using PVSS.Share algorithm. Then the equations in the PVSS.Verify algorithm can be proved to be true as follows.

$$\begin{aligned}
 C'_i &= \text{pk}_i^{p'(i)} = \text{pk}_i^{\tilde{p}(i)} \text{pk}_i^{c \cdot p(i)} = \text{pk}_i^{\tilde{p}(i)} \cdot C_i^c, \forall i \in [1, \dots, n] \\
 \tilde{s} &= \tilde{p}(0) = \text{intpl}(\{(i, p'(i) - c \cdot p(i))\}) \\
 &= \text{intpl}(\{(i, p'(i))\}) - \text{intpl}(\{(i, c \cdot p(i))\}) \\
 &= \text{intpl}(\{(i, \tilde{p}(i))\})
 \end{aligned}$$

Therefore, if the dealer knows the secret s , he can prove it in (C, π) with probability 1.

Lemma 8 (Soundness of Sigma Protocol): If the dealer does not know s , he cannot cheat the verifier successfully.

Proof. It is widely adopted to prove Sigma Protocol **Soundness** by extracting the secret when two accepting proofs with the same commitment and different challenges are given [29]. Denote the two accepting proofs $\pi_1 = (C', c_1, (\tilde{s}_1, \{\tilde{p}_1(i)\}))$ and $\pi_2 = (C', c_2, (\tilde{s}_2, \{\tilde{p}_2(i)\}))$ for the statement C , where $c_1 \neq c_2$. Hence, we have:

$$\begin{cases} C'_i = \text{pk}_i^{\tilde{p}_1(i)} \cdot C_i^{c_1} = \text{pk}_i^{\tilde{p}_1(i) + c_1 \cdot p(i)}, \\ C'_i = \text{pk}_i^{\tilde{p}_2(i)} \cdot C_i^{c_2} = \text{pk}_i^{\tilde{p}_2(i) + c_2 \cdot p(i)}, \end{cases} \forall i \in [1, \dots, n]$$

By dividing the two equations, we get:

$$1_{\mathbb{G}} = \text{pk}_i^{\tilde{p}_2(i) - \tilde{p}_1(i) + (c_2 - c_1) \cdot p(i)}$$

Then $p(i)$ can be calculated as:

$$p(i) = \frac{\tilde{p}_2(i) - \tilde{p}_1(i)}{c_1 - c_2}$$

Further, $s \leftarrow \text{intpl}(\{(i, p(i))\}_{i \in [1, \dots, n]})$ and the secret s can be extracted with (C, π_1, π_2) .

Theorem 11 (Public Verifiability): The messages from the dealer and the shares sent by shareholders are publicly verifiable.

Proof. By Lemma 7 and Lemma 8, the Sigma protocol used in the proposed PVSS provides a proof of knowledge for the dealer, who acts as the prover. Specifically, the output (C, π) of the PVSS.Share algorithm represents a non-interactive proof of knowledge for the secret s or gs . Moreover, the proof π can be verified publicly. Then we consider the public verifiability of shareholders in the reconstruction phase. Since C_i (in C) is already publicly verified and pk_i is publicly known, anyone can determine whether D_i from shareholder P_i is valid or not by $e(D_i, \text{pk}_i) \stackrel{?}{=} e(g, C_i)$, as shown in the PVSS.Recon algorithm.

Lemma 9 (Zero Knowledge of Sigma Protocol): The output of PVSS.Share reveals no information about a shareholder P_i 's $g^{p(i)}$, the dealer's secret gs .

Proof. We introduce a simulator $\mathcal{S}$, taking input a valid statement $C \in \mathbb{G}$ and a challenge $c \in \mathbb{Z}_p$. We prove that $\mathcal{S}$ can produce an accepting proof $(C', c, (\tilde{s}, \{\tilde{p}(i)\}))$ for C . Moreover, the proof should have the same distribution as a transcript generated by a real-world prover and verifier.

In the PVSS.Share algorithm of the real world, an honest prover can always output a transcript $(C', c, (\tilde{s}, \{\tilde{p}(i)\}))$, which is randomly distributed. Then, we argue the output of the simulator $\mathcal{S}$. For $\forall i \in [1 \dots n]$, it firstly calculates :

$$p_t(i) \xleftarrow{R} \mathbb{Z}_p, C_{it} \leftarrow \text{pk}_i^{p_t(i)} \cdot C_i^c$$

and outputs a tuple $(C_{it}, c, p_t(i))$. Notice that the tuple always represents an accepting proof, as required. Further, since c and $p_t(i)$ are randomly distributed in $\mathbb{Z}_p$, it follows that C_{it} is also randomly distributed in $\mathbb{G}$. Summarily, the simulator $\mathcal{S}$ can always output a transcript that is indistinguishable from the output of real-world prover and verifier, meaning that nothing about $p(i)$ is leaked. Hence, the adversary cannot obtain the P_i 's $g^{p(i)}$ without $p(i)$. Besides, it is also infeasible to defer $g^{p(i)}$ using $C'_i = \text{pk}_i^{p(i)} = (g^{p(i)})^{\text{sk}_i}$ under the decisional diffie-hellman assumptions (DDH).

Definition 2 (INDI-Secrecy Game): A PVSS scheme achieves **INDI-Secrecy** if for any polynomial time adversary $\mathcal{A}$ corrupting at most $t-1$ parties, $\mathcal{A}$ has negligible advantage in the following game.

- 1) A challenger $\mathcal{C}$ runs the PVSS.Setup algorithm, creates $(\text{pk}_i, \text{sk}_i)$ for all uncorrupted shareholder P_i and sends the corresponding public keys and all public information to $\mathcal{A}$.
- 2) $\mathcal{A}$ creates secret keys for the $t-1$ corrupted parties and sends the corresponding public keys $\{\text{pk}_i\}$ to the challenger $\mathcal{C}$.
- 3) $\mathcal{C}$ selects two random values $x_0, x_1 \in \mathbb{G}$ and randomly chooses $b \leftarrow \{0, 1\}$. It then runs the PVSS.Share algo-

algorithm with secret x_0 and sends all the output to $\mathcal{A}$, along with x_b .

- 4) $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$.

$\mathcal{A}$'s advantage over the game is defined as $|\Pr[b = b'] - 1/2|$.

Theorem 12 (INDI-Secrecy): The proposed PVSS scheme achieves **INDI-Secrecy**, i.e., for any probabilistic polynomial time adversary $\mathcal{A}$, corrupting fewer than t shareholders, has a negligible advantage in obtaining information about gs .

Proof. By Lemma 9, the Σ -protocol used in the proposed PVSS provides zero knowledge about each shareholder's witness $g^{p(i)}$ for $i \in [1, \dots, n]$ in the PVSS.Share phase. Then, we consider the situation where $\mathcal{A}$ corrupts $t-1$ shareholders. We prove the **INDI-Secrecy** property by analyzing the security game defined in Definition 2 based on DDH assumption. We argue that, if there exists an adversary $\mathcal{A}$ which can break the **INDI-Secrecy** property of our PVSS scheme, then there exists an adversary $\mathcal{A}_{DDH}$ which can use $\mathcal{A}$ to break DDH assumption. Without loss of generality, denote the first $t-1$ shareholders, i.e., $[P_1, \dots, P_{t-1}]$, as the corrupted parties.

Let $(h, h^\alpha, h^\beta, h^\gamma)$ be an instance of the DDH problem. Then, $\mathcal{A}_{DDH}$ using $\mathcal{A}$ can simulate a security game as follows:

- 1) The challenger $\mathcal{C}$ sets $g = h^\alpha$. For $t \leq i \leq n$, $\mathcal{A}_{DDH}$ randomly choose $r_i \xleftarrow{R} \mathbb{Z}_q$ and sends the values $\text{pk}_i = h^{r_i}$ to $\mathcal{A}$.
- 2) For $1 < i \leq t-1$, $\mathcal{A}$ randomly chooses $\text{sk}_i \xleftarrow{R} \mathbb{Z}_q$ and sets $\text{pk}_i = g^{\text{sk}_i}$, then it sends these to the challenger $\mathcal{C}$.
- 3) For $1 < i \leq t-1$, $\mathcal{C}$ chooses uniformly random values $s_i \xleftarrow{R} \mathbb{Z}_p$ and sets $C_i^* = \text{pk}_i^{s_i}$.

For $t \leq i \leq n$, $\mathcal{C}$ generates $h_i^{p(i)}$ and determines $p(x)$ as a polynomial of degree at most $t-1$ where $p(i) = s_i$ for $1 \leq i \leq t-1$ and $p(0) = \beta$. Note that $\mathcal{A}_{DDH}$ knows h^β and h^{s_i} for $1 \leq i \leq t-1$. So $\mathcal{A}_{DDH}$ can use Lagrange interpolation to compute $S_i^* = h^{s_i} = h^{p(i)}$ for $t \leq i \leq n$ and it also generates shares $C_i^* = (S_i^*)^{r_i} = \text{pk}_i^{s_i}$.

To simulate the NIZK proof π , $\mathcal{A}_{DDH}$ randomly chooses $c, \tilde{s} \in \mathbb{Z}_p$ and constructs a random polynomial $\tilde{p}(x)$ of degree $t-1$ such that $\tilde{p}(0) = \tilde{s}$. Then, $\mathcal{A}_{DDH}$ sets $C'_i = (C_i^*)^c \cdot \text{pk}_i^{\tilde{p}(i)}$ for $1 \leq i \leq n$ and programs the random oracle $H(\cdot)$ so that $c = H(\{C_i^*\}, \{C'_i\})$.

Finally, $\mathcal{C}$ sends $(C^* = \{C_i^*\}_{i \in [1, \dots, n]}, \pi)$ together with h^γ to $\mathcal{A}$, where h^γ plays the role of x_b in the INDI-Secrecy Game (defined by Definition 2).

- 4) $\mathcal{A}$ makes a guess of b' .

If $b' = 0$, $\mathcal{A}_{DDH}$ guess that $\gamma = \alpha \cdot \beta$, if $b' = 1$, $\mathcal{A}_{DDH}$ guess that γ is a random element. The information sent to $\mathcal{A}$ is distributed exactly like a sharing of the value $g^\beta = h^{\alpha \cdot \beta}$. If h^γ sent to $\mathcal{A}$ is the secret shared by PVSS, $\gamma = \alpha \cdot \beta$. So the advantage of $\mathcal{A}_{DDH}$ is the same as the advantage of $\mathcal{A}$.